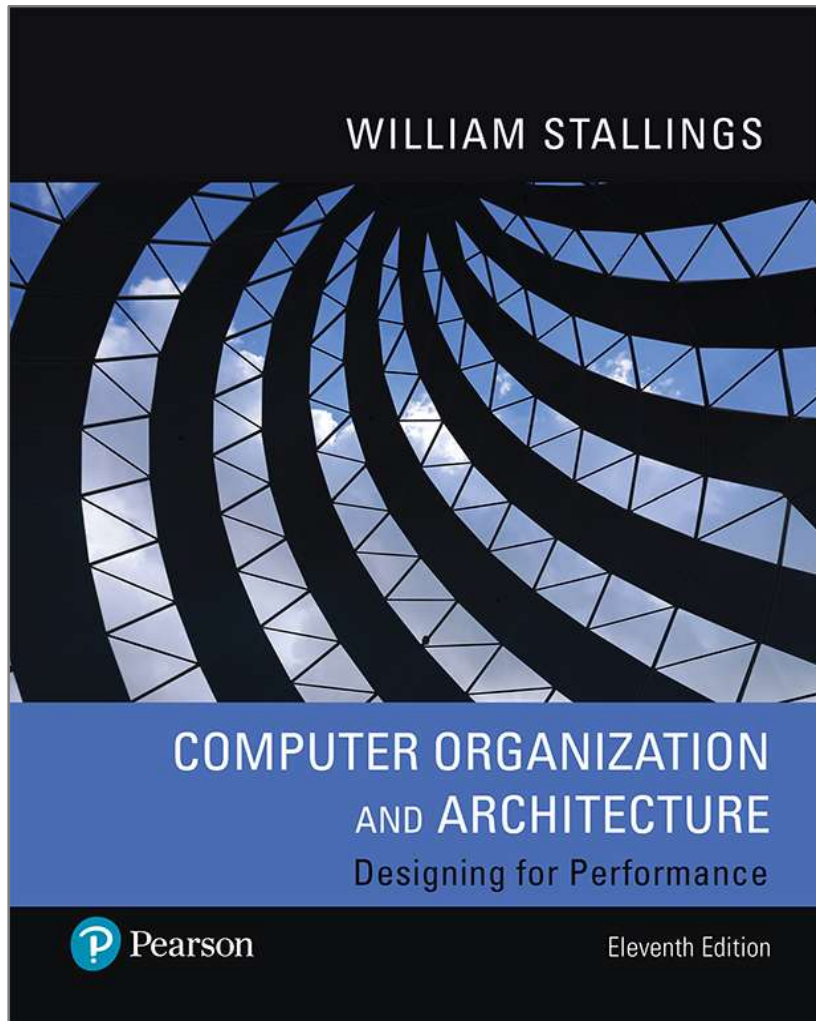


Computer Organization and Architecture

Designing for Performance

11th Edition



Chapter 4

The Memory Hierarchy: Cache Memory

Principle of Locality (1 of 2)

- Also referred to as the *locality of reference*
- Reflects the observation that during the course of execution of a program, memory references by the processor tend to cluster
- Locality is based on three assertions:
 - During any interval of time, a program references memory location non-uniformly
 - As a function of time, the probability that a given unit of memory is referenced tends to change slowly
 - The correlation between immediate past and immediate future memory reference patterns is high and tapers off as the time interval increases

Principle of Locality (2 of 2)

- Two forms of locality
 - Temporal locality
 - Refers to the tendency of a program to reference in the near future those units of memory referenced in the recent past
 - Constants, temporary variables, and working stacks are also constructs that lead to this principle
 - Spatial locality
 - Refers to the tendency of a program to reference units of memory whose addresses are near one another
 - Also reflects the tendency of a program to access data locations sequentially, such as when processing a table of data

Library Analogy Explained

- Registers $\leftrightarrow$ books on your desk
 - Actively being used, small capacity
- Caches $\leftrightarrow$ bookshelves
 - Moderate capacity, pretty fast to access
- Main memory $\leftrightarrow$ library
 - Big, holds almost all data, but slow
- Disk (virtual memory) $\leftrightarrow$ inter-library loan
 - Very slow, but hopefully really uncommon

Table 4.1

Key Characteristics of Computer Memory Systems

Location Internal (e.g., processor registers, cache, main memory) External (e.g., optical disks, magnetic disks, tapes)	Performance Access time Cycle time Transfer rate
Capacity Number of words Number of bytes	Physical Type Semiconductor Magnetic Optical Magneto-optical
Unit of Transfer Word Block	Physical Characteristics Volatile/nonvolatile Erasable/nonerasable
Access Method Sequential Direct Random Associative	Organization Memory modules

Characteristics of Memory Systems

- Location
 - Refers to whether memory is internal and external to the computer
 - Internal memory is often equated with main memory
 - Processor requires its own local memory, in the form of registers
 - Cache is another form of internal memory
 - External memory consists of peripheral storage devices that are accessible to the processor via I/O controllers
- Capacity
 - Memory is typically expressed in terms of bytes
- Unit of transfer
 - For internal memory the unit of transfer is equal to the number of electrical lines into and out of the memory module

Capacity and Performance:

The two most important characteristics of memory

Three performance parameters are used:

Access time (latency)

- For random-access memory it is the time it takes to perform a read or write operation
- For non-random-access memory it is the time it takes to position the read-write mechanism at the desired location

Memory cycle time

- Access time plus any additional time required before second access can commence
- Additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively
- Concerned with the system bus, not the processor

Transfer rate

- The rate at which data can be transferred into or out of a memory unit
- For random-access memory it is equal to $1/(\text{cycle time})$

Memory Hierarchy

- Design constraints on a computer's memory can be summed up by three questions:
 - How much, how fast, how expensive
- There is a trade-off among capacity, access time, and cost
 - Faster access time, greater cost per bit
 - Greater capacity, smaller cost per bit
 - Greater capacity, slower access time

Figure 4.6

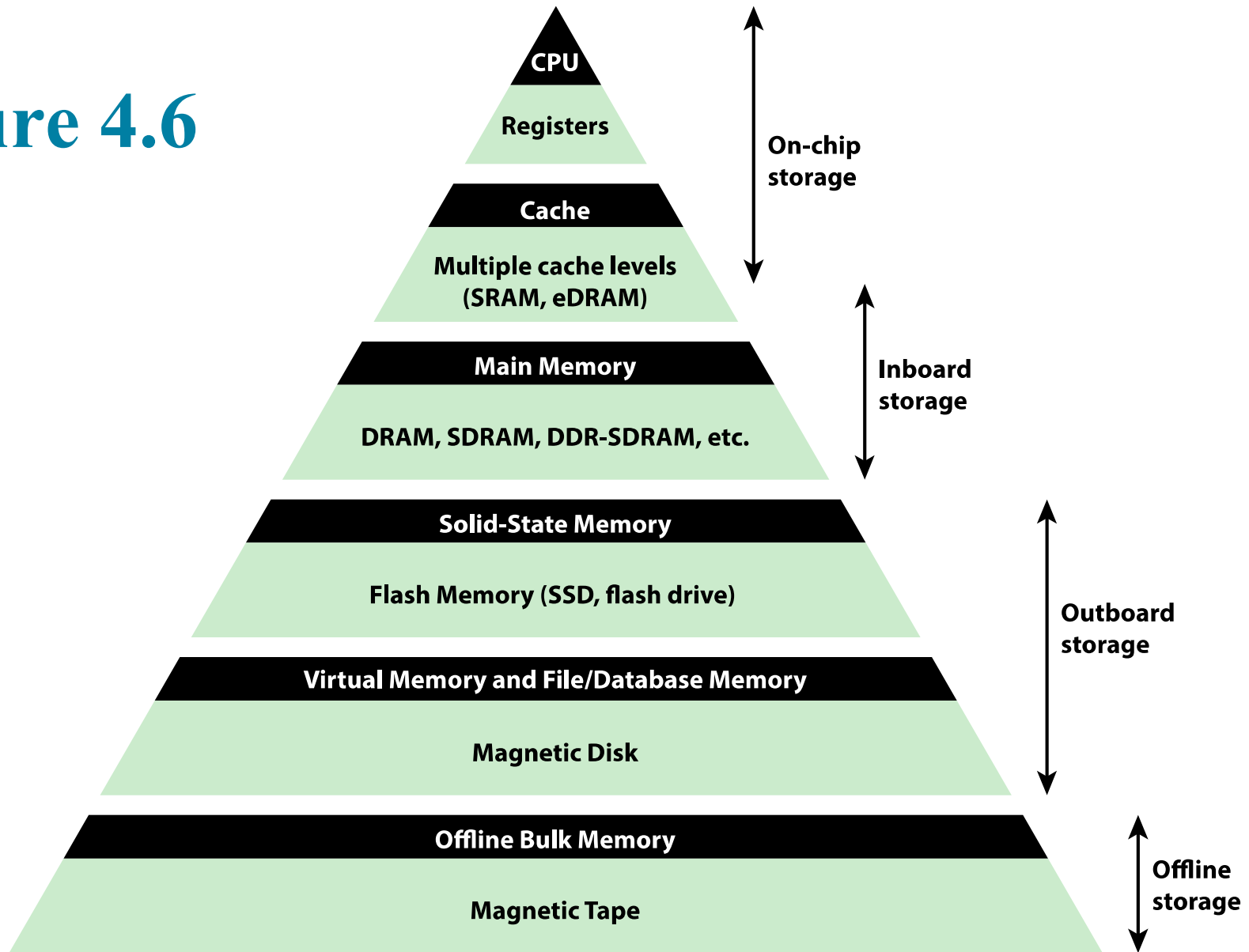


Figure 4.6 The Memory Hierarchy

Table 4.2

Characteristics of Memory Devices in a Memory Architecture

Memory level	Typical technology	Unit of transfer with next larger level (typical size)	Managed by
Registers	CMOS	Word (32 bits)	Compiler
Cache	Static RAM (SRAM); Embedded dynamic RAM (eDRAM)	Cache block (32 bytes)	Processor hardware
Main memory	DRAM	Virtual memory page (1 kB)	Operating system (OS)
Secondary memory	Magnetic disk	Disk sector (512 bytes)	OS/user
Offline bulk memory	Magnetic tape		OS/User

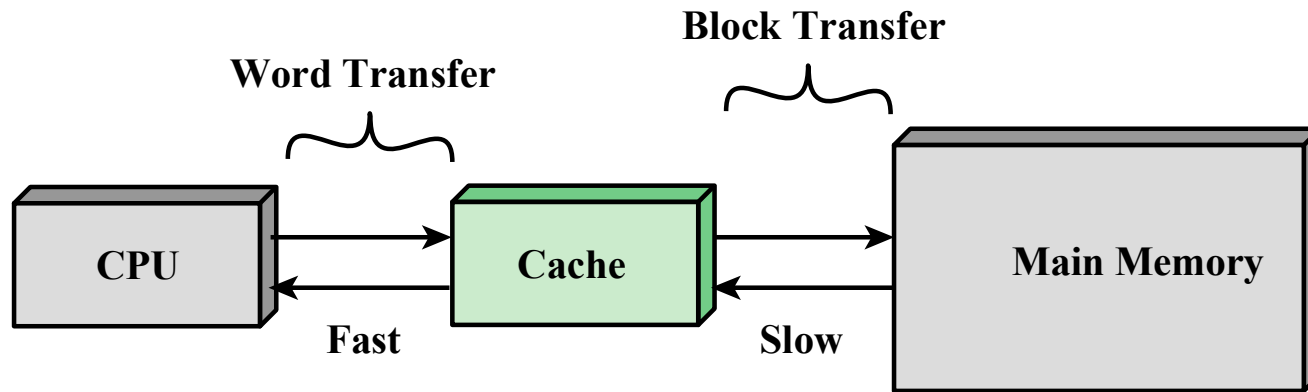
Table 4.2 Characteristics of Memory Devices in a Memory Architecture

Why Cache?

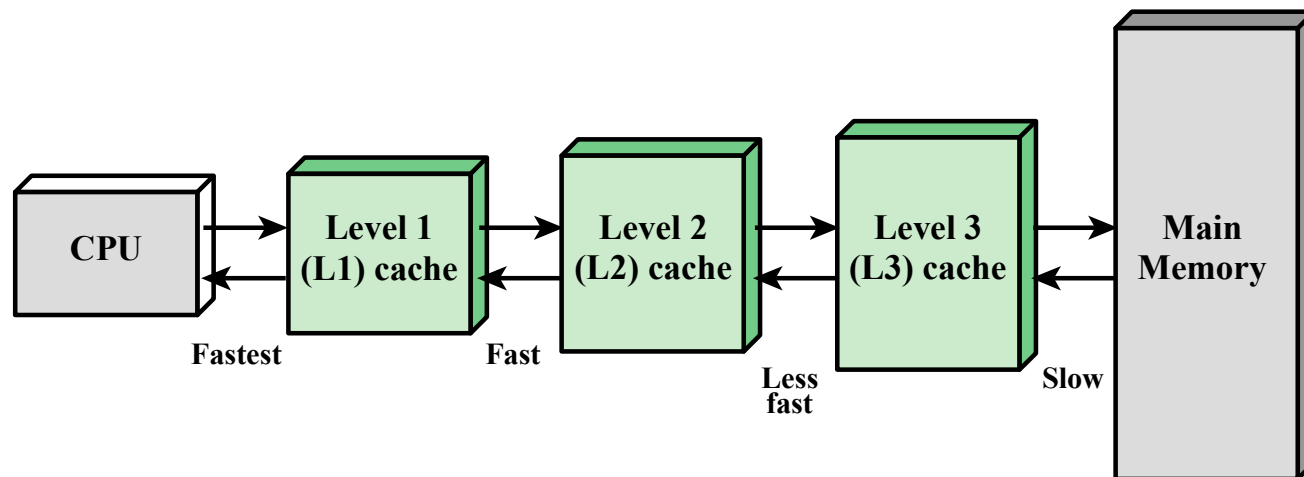
- Analysis several programmed show that computer memory tend to refer into one particular area.
- This phenomenon is named as *locality of reference*.
- This phenomenon give strong argument so that data those involve in this phenomenon is housed into memory high speed(**cache memory**), which it can speed up whole system access time (**overall access time**).
- Example: **Loop execution or table, array referring.**

Figure 5.1

Cache and Main Memory



(a) Single cache



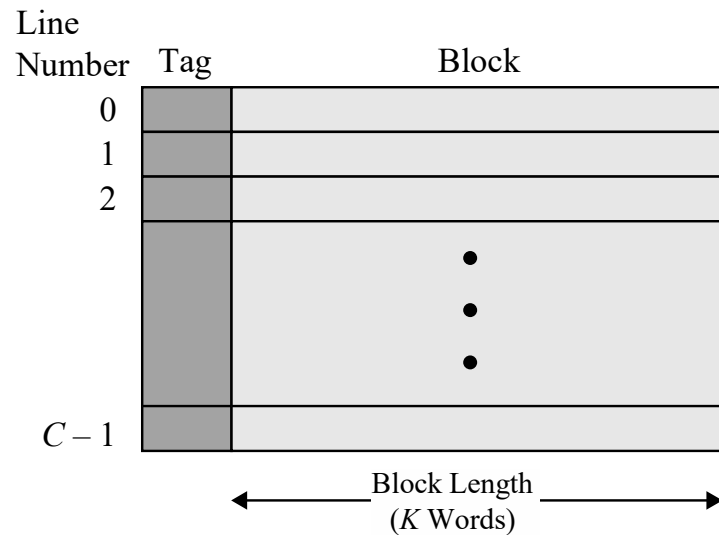
(b) Three-level cache organization

Cache Memory Principles

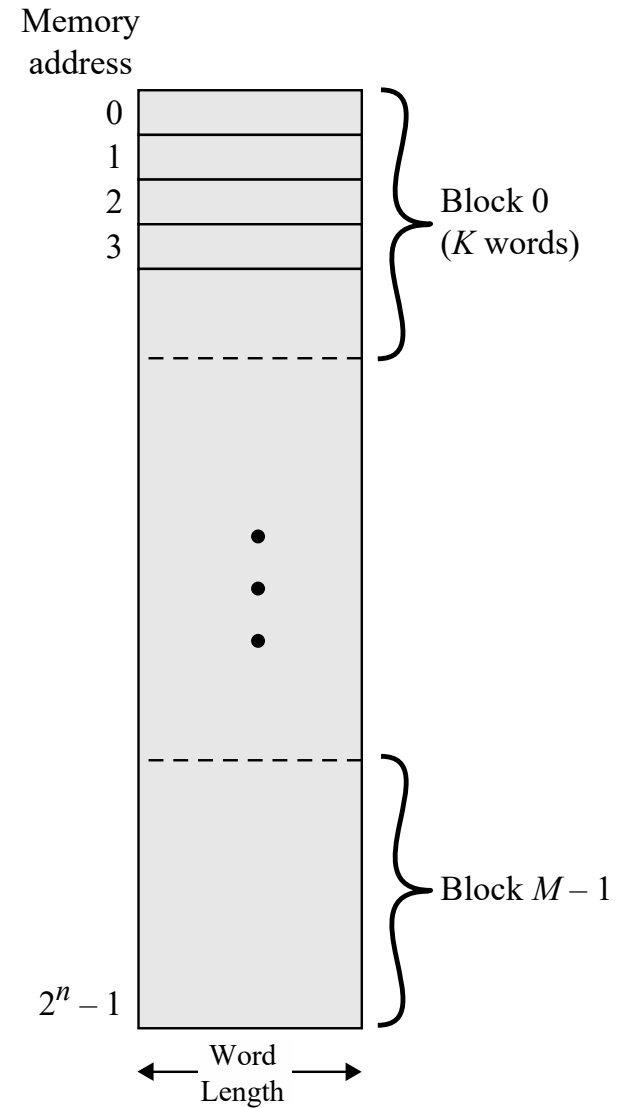
- Block
 - The minimum unit of transfer between cache and main memory
- Frame
 - To distinguish between the data transferred and the chunk of physical memory, the term frame, or block frame, is sometimes used with reference to caches
- Line
 - A portion of cache memory capable of holding one block, so-called because it is usually drawn as a horizontal object
- Tag
 - A portion of a cache line that is used for addressing purposes
- Line size
 - The number of data bytes, or block size, contained in a line

Figure 5.2

Cache/Main Memory Structure



(a) Cache



(b) Main memory

Figure 5.3

Cache Read Operation

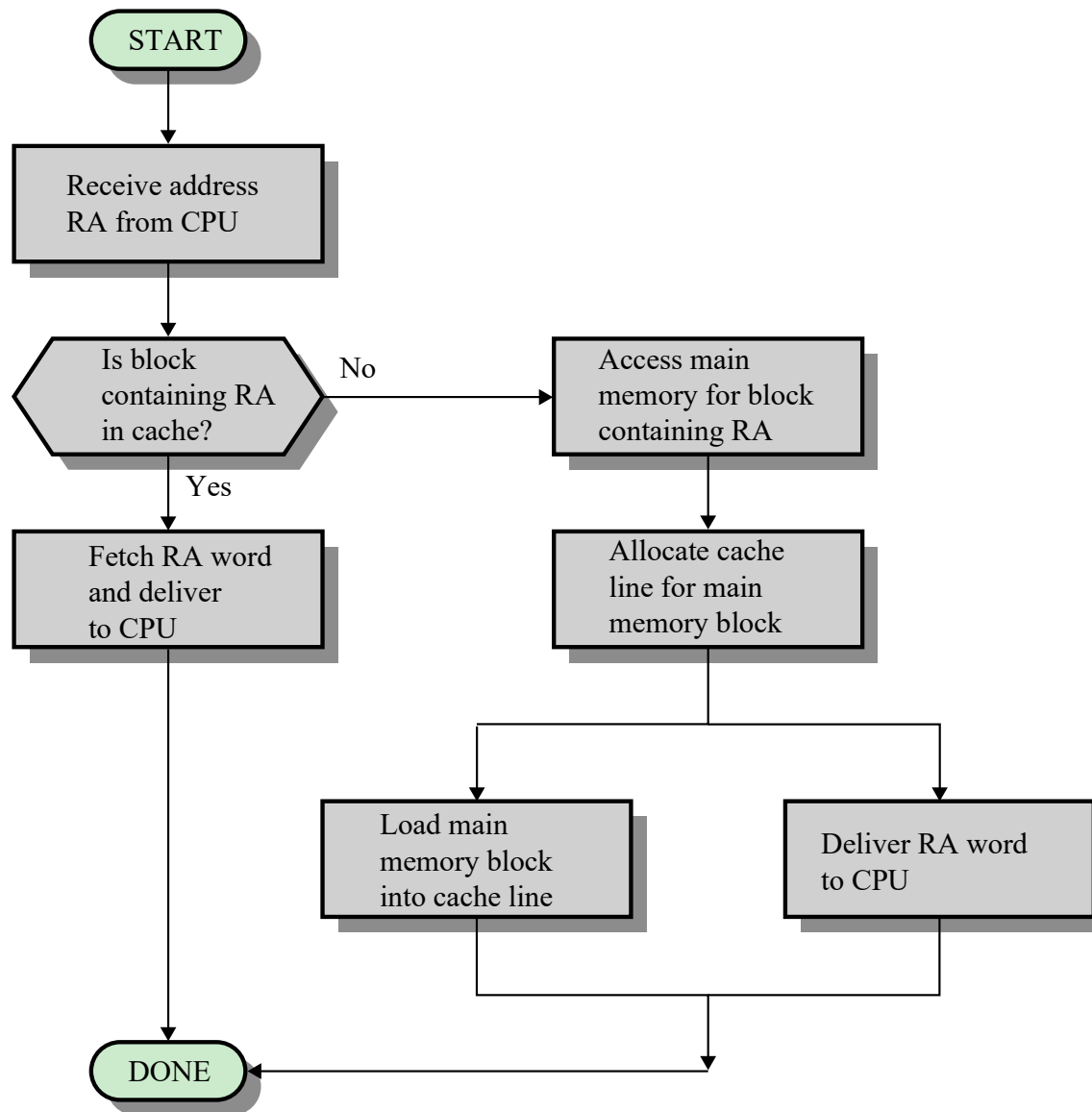
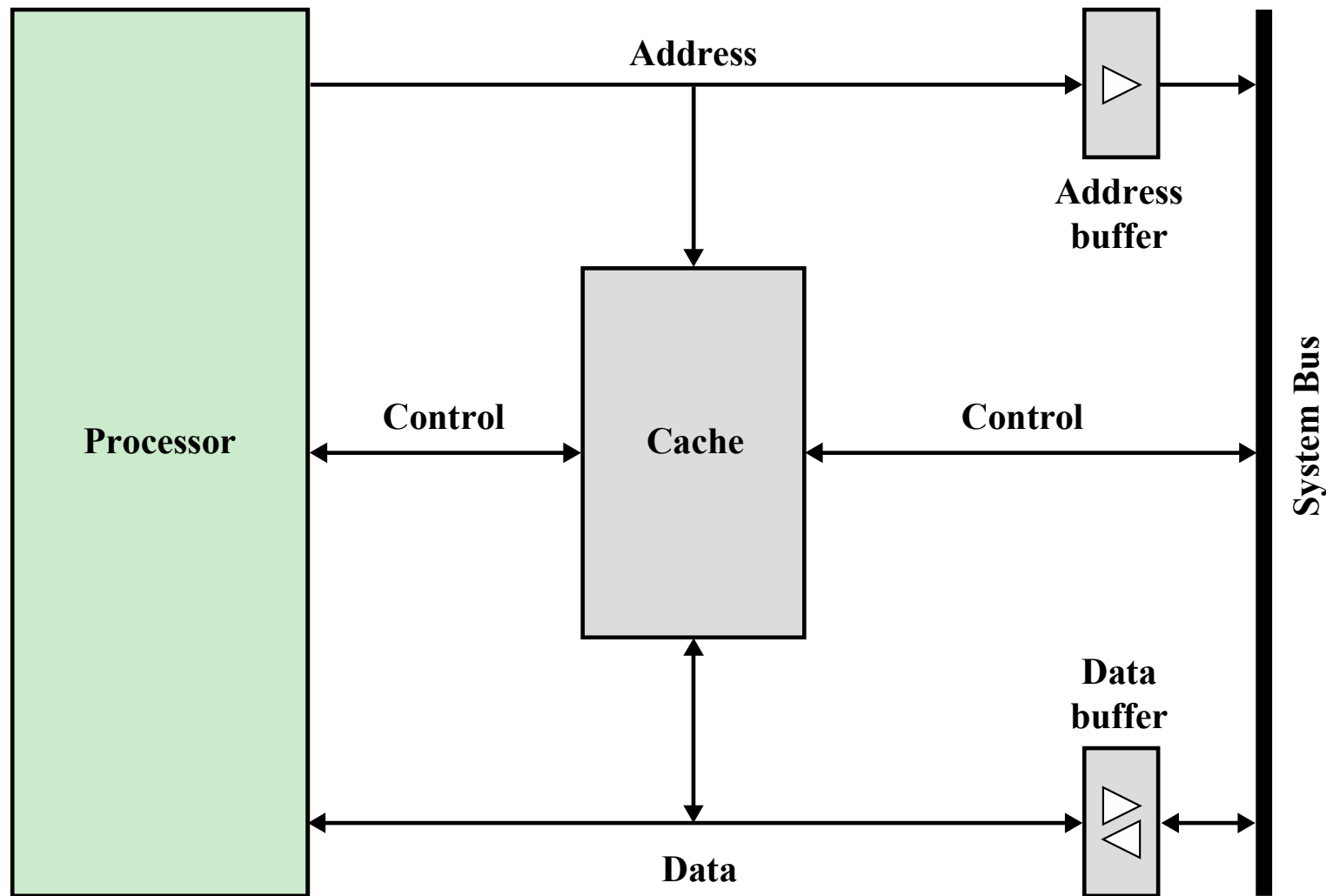


Figure 5.4

Typical Cache Organization



Terminologi Cache

- **Hit**

- an access where the data is found in the cache

- **Miss**

- an access which is not found in the cache

Cache Performance

- **Hit time:** Time to access the cache
- **Miss time:** Time to process a cache miss
 - Move data from lower level memory to the cache and CPU
- **Hit ratio:** % of time the data is found in the cache
- **Miss ratio:** (1 - hit ratio)
- **Cache block size or cache line size:** the amount of data that gets transferred on a cache miss
- **Effective access time:** $(\text{Hit Ratio} * \text{Hit Time}) + (\text{Miss Ratio} * \text{Miss Time})$

Can Cache Increases Performance Of The System?

- Take the following access time of A computer system, ($T_{ma} = \mathbf{500ns}$)
- Based on the above, the overall access time to the memory with no cache is,
 - $T_{acc} = T_{ma} = \mathbf{500ns}$
- If the above system has a cache with an access time of $\mathbf{50ns}$ (T_{ca}) and a hit ratio of $\mathbf{90\%}$, the overall access time is,
- $T_{acc} = (T_{ca} \times 0.90) + (T_{ma} \times (1-0.9))$
- $T_{acc} = (50 \times 0.90) + (500 \times 0.10)$
- $T_{acc} = (45) + (50)$
- $T_{acc} = \mathbf{95ns}$

Table 5.1

Elements of Cache Design

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Cache Size

- Preferable for the size of the cache to be:
 - Small enough so that the overall average cost per bit is close to that of main memory alone
 - Large enough so that the overall average access time is close to that of the cache alone
- Motivations for minimizing cache size:
 - The larger the cache, the larger the number of gates involved in addressing the cache resulting in large caches being slightly slower than small ones
 - The available chip and board area also limits cache size
- Because the performance of the cache is very sensitive to the nature of the workload, it is impossible to arrive at a single “optimum” cache size

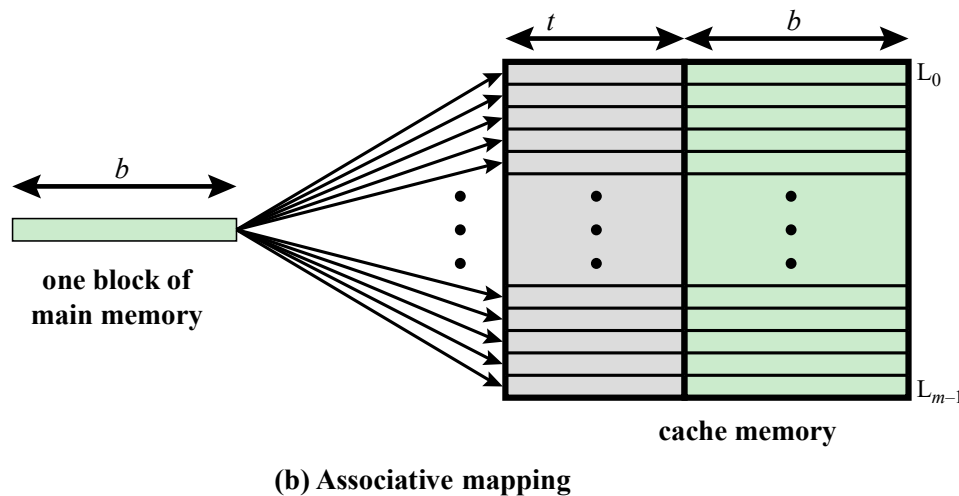
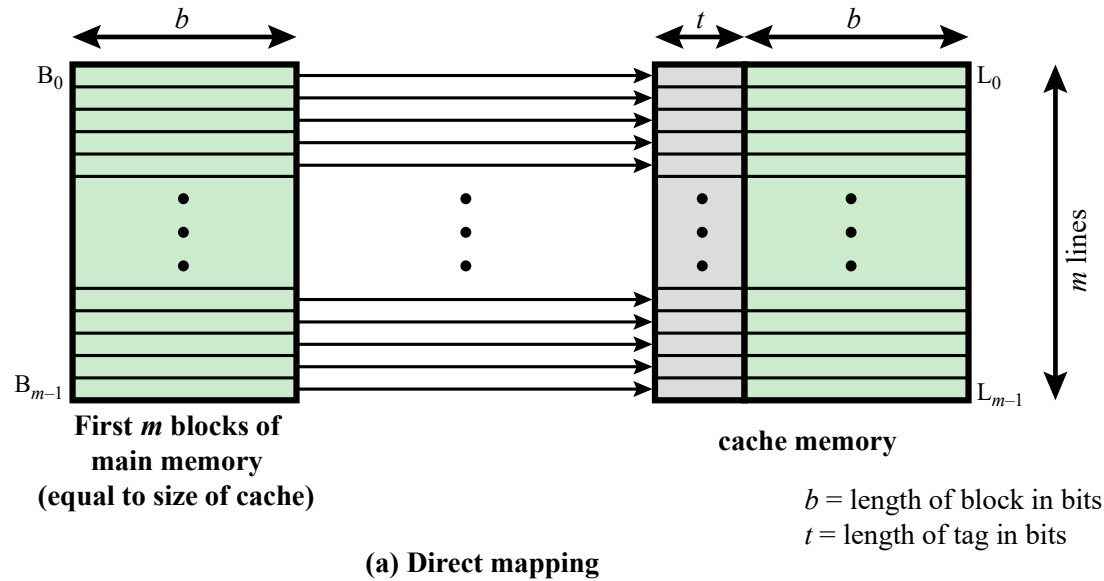
Table 5.3

Cache Access Methods

Method	Organization	Mapping of Main Memory Blocks to Cache	Access using Main Memory Address
Direct Mapped	Sequence of m lines	Each block of main memory maps to one unique line of cache.	<i>Line</i> portion of address used to access cache line; <i>Tag</i> portion used to check for hit on that line.
Fully Associative	Sequence of m lines	Each block of main memory can map to any line of cache.	<i>Tag</i> portion of address used to check every line for hit on that line.
Set Associative	Sequence of m lines organized as v sets of k lines each ($m = v \times k$)	Each block of main memory maps to one unique cache set.	<i>Line</i> portion of address used to access cache set; <i>Tag</i> portion used to check every line in that set for hit on that line.

Figure 5.6

Mapping from Main Memory to Cache: Direct and Associative



Direct Mapping

- Each block of main memory maps to only one cache line
- Address is in two parts
- Least Significant **w** bits identify **unique word**
- Most Significant **s** bits specify **one memory block**
- The MSBs are split into a cache line field r and a tag of $s-r$ (most significant)

Direct Mapping Address Structure

Tag s-r	Line or Slot r	Word w
8	14	2

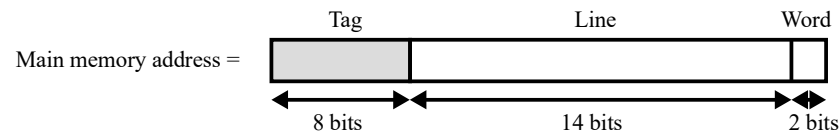
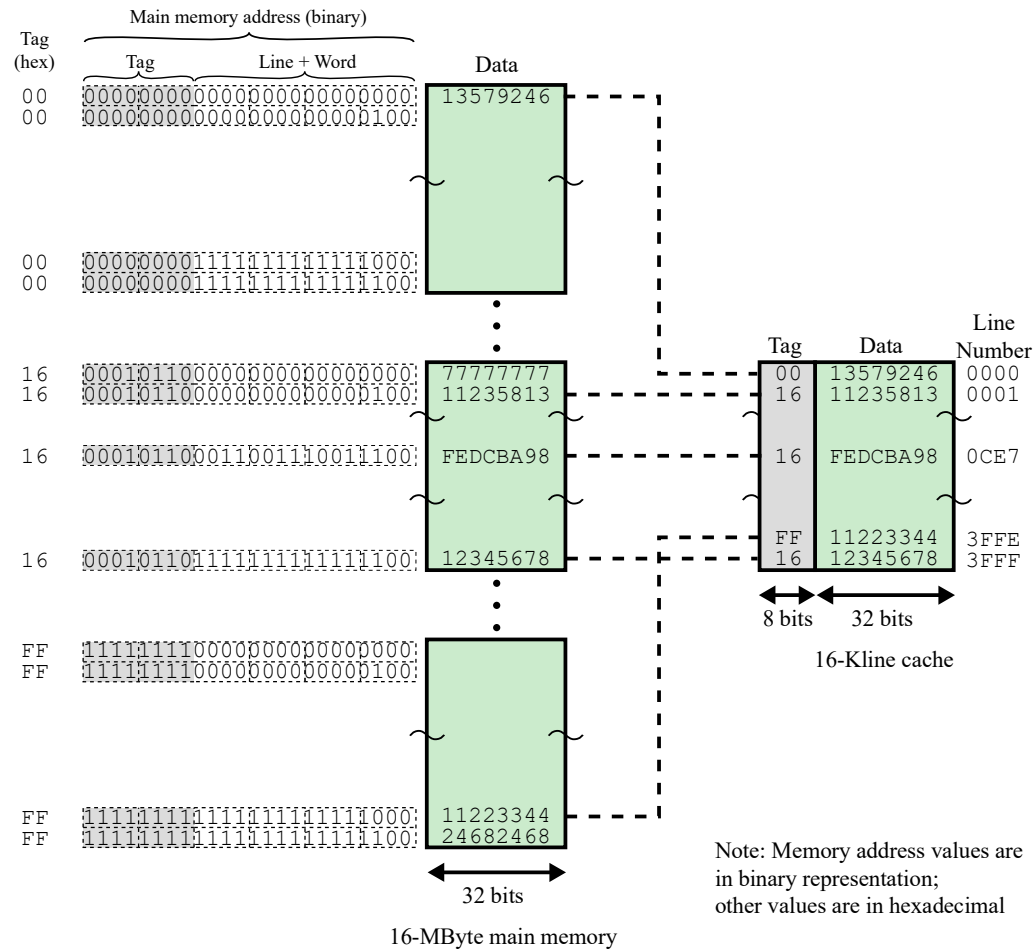
- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
 - 8 bit tag (=22-14)
 - 14 bit slot or line
- **No two blocks in the same line have the same Tag field**
- Check contents of cache by **finding line** and **checking Tag**

Direct Mapping Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = $m = 2^r$
- Size of cache = 2^{r+w}
- Size of tag = $(s - r)$ bits

Figure 5.8

Direct Mapping Example



Associative Mapping

- A main memory block can load into any line of cache
- Memory address is interpreted as tag and word
- Tag uniquely identifies block of memory
- Every line's tag is examined for a match
- Cache searching **gets expensive**

Associative Mapping Address Structure

Tag 22 bit	Word 2 bit
------------	---------------

- 22 bit tag stored with each 32 bit block of data
- Compare **tag field with tag entry in cache to check for hit**
- Least significant 2 bits of address identify which 8 bit word is required from 32 bit data block
- e.g.

Address	Tag	Data	Cache line
– FFFFFC	3FFFFFF	24682468	3FFF

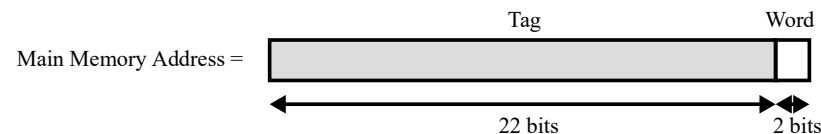
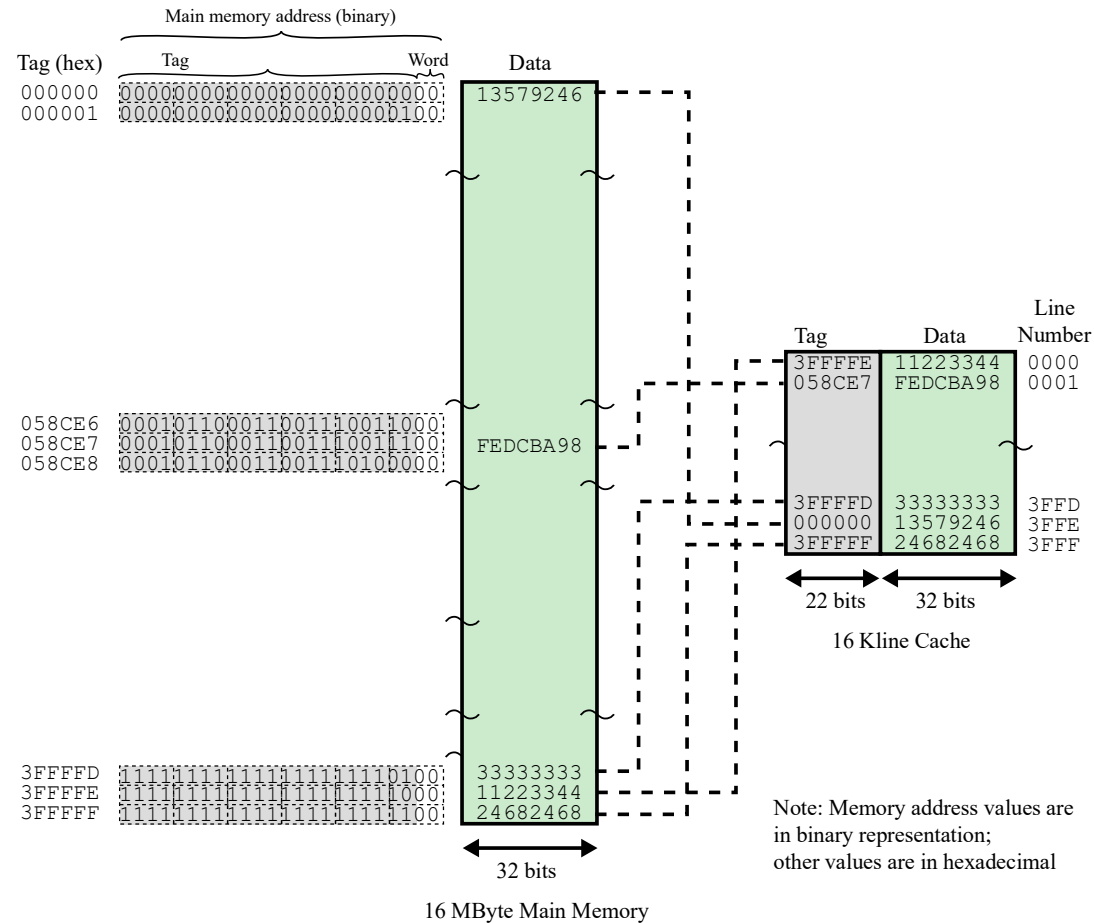
Associative Mapping

Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = undetermined
(cache size/block size)
- Size of tag = s bits

Figure 5.11

Associative Mapping Example

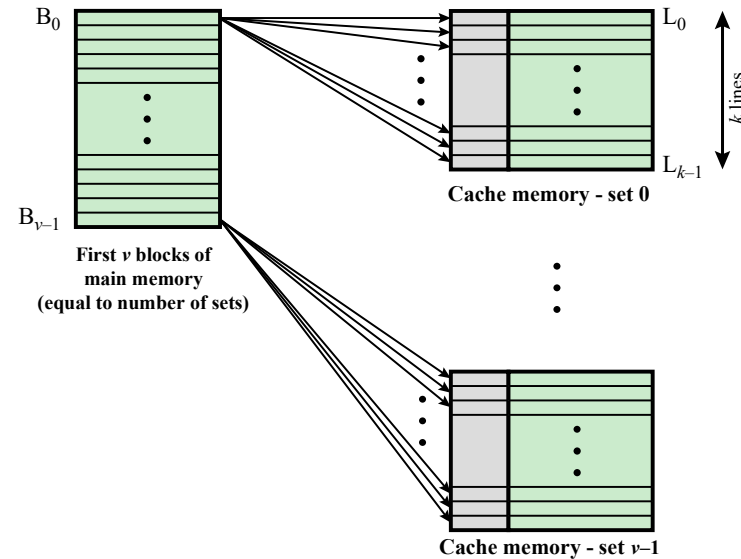


Set Associative Mapping

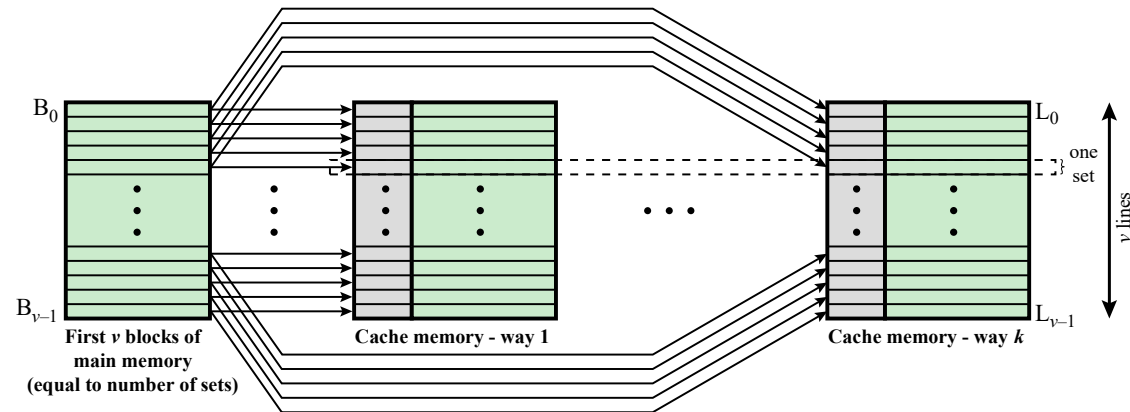
- Compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages
- Cache consists of a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
- e.g. 2 lines per set
 - 2 way associative mapping
 - A given block can be in one of 2 lines in only one set

Figure 5.12

Mapping from Main Memory to Cache: k-Way Set Associative



(a) v associative-mapped caches



(b) k direct-mapped caches

Set Associative Mapping Address Structure

Tag 9 bit	Set 13 bit	Word 2 bit
-----------	------------	---------------

- Use **set field** to determine cache set to look in
- Compare **tag field** to see if we have a hit
- e.g

– Address	Tag	Data	Set number
– 167FFC	02C	12345678	1FFF
– FFFFFFF8	1FF	11223344	1FFE

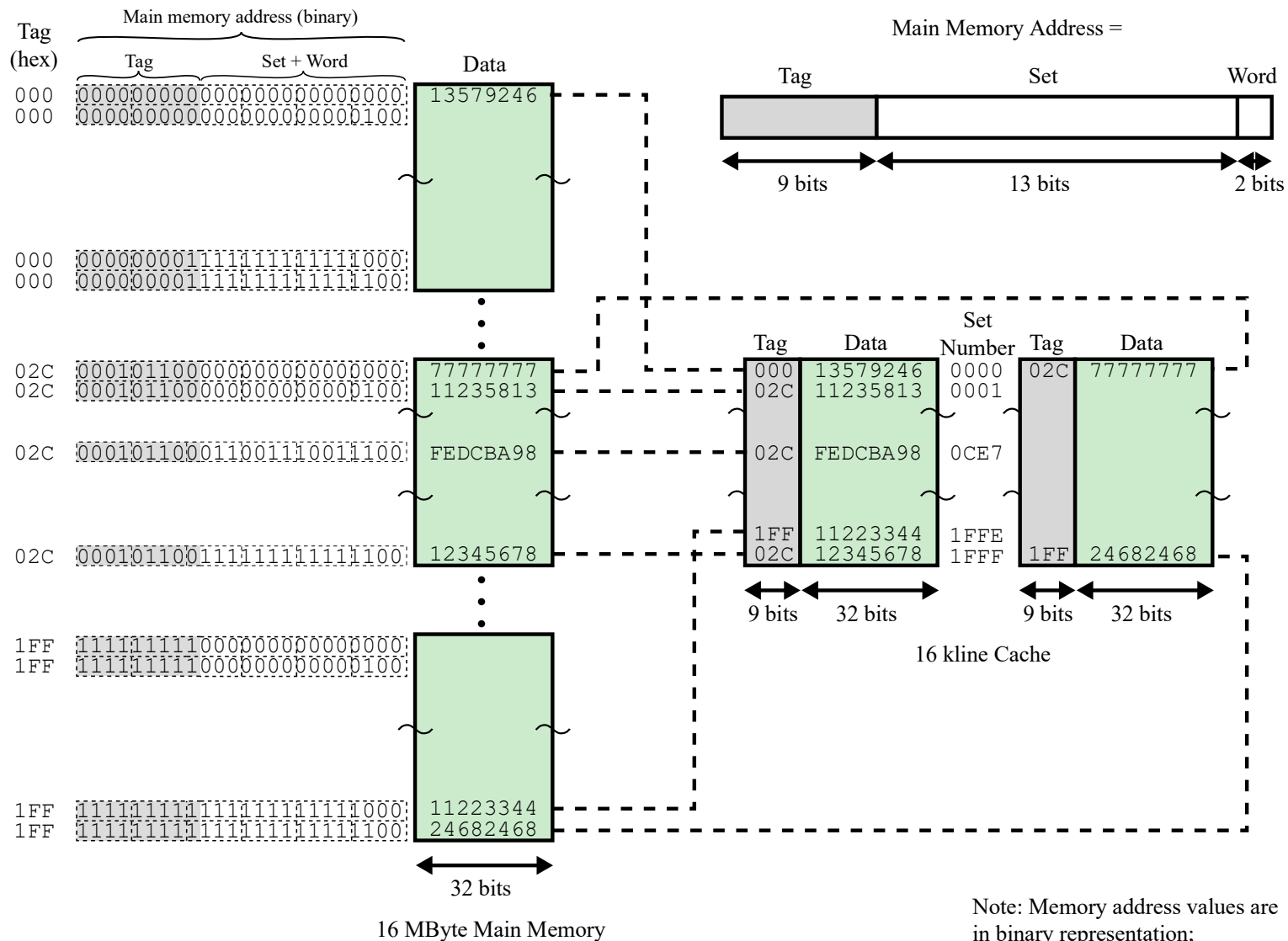
Set Associative Mapping

Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = 2^s
- Number of lines in set = k
- Number of sets = $v = 2^d$
- Number of lines in cache = $k v = k * 2^d$
- Size of tag = $(s - d)$ bits

Figure 5.14

Two-Way Set-Associative Mapping Example



Replacement Algorithms

- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced
- For direct mapping there is only one possible line for any particular block and no choice is possible
- For the associative and set-associative techniques a replacement algorithm is needed
- To achieve high speed, an algorithm must be implemented in hardware

The most common replacement algorithms are:

- Least recently used (LRU)
 - Most effective
 - Replace that block in the set that has been in the cache longest with no reference to it
 - Because of its simplicity of implementation, LRU is the most popular replacement algorithm
- First-in-first-out (FIFO)
 - Replace that block in the set that has been in the cache longest
 - Easily implemented as a round-robin or circular buffer technique
- Least frequently used (LFU)
 - Replace that block in the set that has experienced the fewest references
 - Could be implemented by associating a counter with each line

Write Policy

When a block that is resident in the cache is to be replaced there are two cases to consider:



If the old block in the cache has not been altered then it may be overwritten with a new block without first writing out the old block



If at least one write operation has been performed on a word in that line of the cache then main memory must be updated by writing the line of cache out to the block of memory before bringing in the new block

There are two problems to contend with:



More than one device may have access to main memory



A more complex problem occurs when multiple processors are attached to the same bus and each processor has its own local cache - if a word is altered in one cache it could conceivably invalidate a word in other caches

Write Through and Write Back

- Write through
 - Simplest technique
 - All write operations are made to main memory as well as to the cache
 - The main disadvantage of this technique is that it generates substantial memory traffic and may create a bottleneck
- Write back
 - Minimizes memory writes
 - Updates are made only in the cache
 - Portions of main memory are invalid and hence accesses by I/O modules can be allowed only through the cache
 - This makes for complex circuitry and a potential bottleneck

Write Miss Alternatives

- There are two alternatives in the event of a write miss at a cache level:
 - Write allocate
 - The block containing the word to be written is fetched from main memory (or next level cache) into the cache and the processor proceeds with the write cycle
 - No write allocate
 - The block containing the word to be written is modified in the main memory and not loaded into the cache
- Either of these policies can be used with either write through or write back
- No write allocate is most commonly used with write through
- Write allocate is most commonly used with write back

Cache Coherency

- A new problem is introduced in a bus organization in which more than one device has a cache and main memory is shared
- If data in one cache are altered, this invalidates not only the corresponding word in main memory, but also that same word in other caches
- Even if a write-through policy is used, the other caches may contain invalid data
- Possible approaches to cache coherency include:
 - **Bus watching with write through**
 - Each cache controller monitors the address lines to detect write operations to memory by other bus masters
 - If another master writes to a location in shared memory that also resides in the cache memory, the cache controller invalidates that cache entry
 - This strategy depends on the use of a write-through policy by all cache controllers
 - **Hardware transparency**
 - Additional hardware is used to ensure that all updates to main memory via cache are reflected in all caches
 - If one processor modifies a word in its cache, this update is written to main memory
 - **Noncacheable memory**
 - Only a portion of main memory is shared by more than one processor, and this is designated as noncacheable
 - All accesses to shared memory are cache misses, because the shared memory is never copied into the cache
 - The noncacheable memory can be identified using chip-select logic or high-address bits

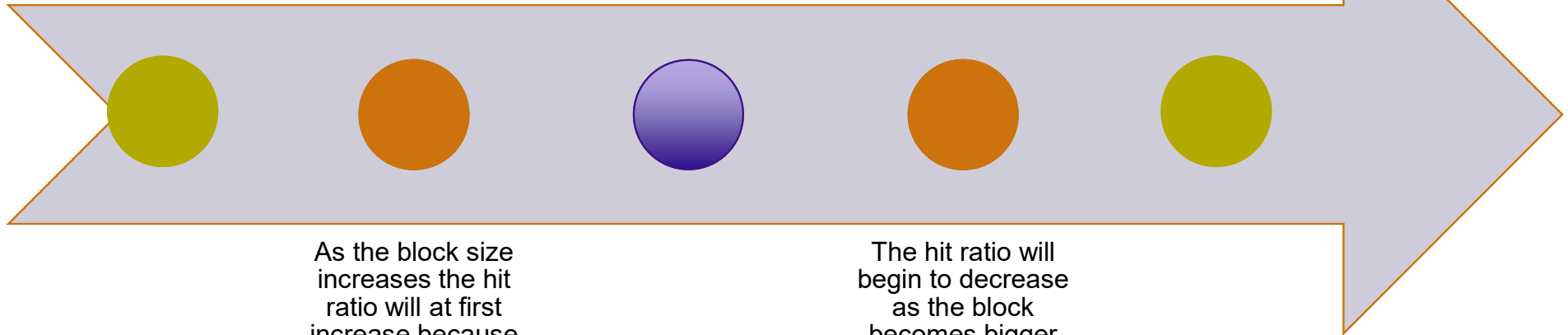
Line Size

When a block of data is retrieved and placed in the cache not only the desired word but also some number of adjacent words are retrieved

As the block size increases more useful data are brought into the cache

Two specific effects come into play:

- Larger blocks reduce the number of blocks that fit into a cache
- As a block becomes larger each additional word is farther from the requested word



As the block size increases the hit ratio will at first increase because of the principle of locality

The hit ratio will begin to decrease as the block becomes bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced

Multilevel Caches

- As logic density has increased it has become possible to have a cache on the same chip as the processor
- The on-chip cache reduces the processor's external bus activity and speeds up execution time and increases overall system performance
 - When the requested instruction or data is found in the on-chip cache, the bus access is eliminated
 - On-chip cache accesses will complete appreciably faster than would even zero-wait state bus cycles
 - During this period the bus is free to support other transfers
- Two-level cache:
 - Internal cache designated as level 1 (L1)
 - External cache designated as level 2 (L2)
- Potential savings due to the use of an L2 cache depends on the hit rates in both the L1 and L2 caches
- The use of multilevel caches complicates all of the design issues related to caches, including size, replacement algorithm, and write policy

Unified Versus Split Caches

- Has become common to split cache:
 - One dedicated to instructions
 - One dedicated to data
 - Both exist at the same level, typically as two L1 caches
- Advantages of unified cache:
 - Higher hit rate
 - Balances load of instruction and data fetches automatically
 - Only one cache needs to be designed and implemented
- Trend is toward split caches at the L1 and unified caches for higher levels
- Advantages of split cache:
 - Eliminates cache contention between instruction fetch/decode unit and execution unit
 - Important in pipelining

Inclusion Policy

- **Inclusive policy**
 - Dictates that a piece of data in one cache is guaranteed to be also found in all lower levels of caches
 - Advantage is that it simplifies searching for data when there are multiple processors in the computing system
 - This property is useful in enforcing cache coherence
- **Exclusive policy**
 - Dictates that a piece of data in one cache is guaranteed not to be found in all lower levels of caches
 - The advantage is that it does not waste cache capacity since it does not store multiple copies of the same data in all of the caches
 - The disadvantage is the need to search multiple cache levels when invalidating or updating a block
 - To minimize the search time, the highest-level tag sets are typically duplicated at the lowest cache level to centralize searching
- **Noninclusive policy**
 - With the noninclusive policy a piece of data in one cache may or may not be found in lower levels of caches
 - As with the exclusive policy, this policy will generally maintain all higher-level cache sets at the lowest cache level

Summary

Chapter 4

- Cache memory principles
- Cache performance modules
 - Cache timing model
 - Design option for improving performance

Cache Memory

- Elements of cache design
 - Cache addresses
 - Cache size
 - Logical cache organization
 - Replacement algorithms
 - Write policy
 - Line size
 - Number of caches
 - Inclusion policy

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructions in teaching their courses and assessing student learning. dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.